

filesystem size of 16 TB and a maximum file size of 2 TB. Since ext4 has 48-bit block addressing, it will have a maximum filesystem size of 1 EB and a maximum file size of 16 TB. This is the simple math: 1 EB = 1,048,576 TB (1 EB = 1024 PB, 1 PB = 1024 TB, 1 TB = 1024 GB). A future update should implement 64-bit block addressing.

Unlimited sub-directories

As opposed to the maximum limit of 32,000 sub-directories under a single directory in ext3, ext4 allows a theoretically unlimited number of sub-directories under the same directory. This is limited only by the available disk space and not by the ext4 implementation.

Extents

Extents is a way to efficiently represent a large contiguous file by reducing the meta-data needed to address large files. An extent is basically a bunch of physical blocks. It is useful if, for instance, you have a 200 MB file. The total number of blocks is 51,200, considering a 4 KB block size. Thus, to track the data stored in each block, we need 51,200 mappings! With extents, we can map the whole file into possibly two extents. This reduces meta data updates while writing files, and thus reduces file system overheads.

The multiple block allocator

Once a *write()* has been invoked, sufficient number of blocks need to be allocated from the pool of free blocks to store the data. ext3 allocates only one block at a time. If you consider a 200 MB file mentioned in the previous point, then the block allocator will be invoked 51,200 times! This is obviously inefficient, and since the block allocator has no idea about the total number of blocks, it cannot decide on an optimal block allocation strategy.

ext4 uses a 'multi-block allocator' (mballoc) which allocates many blocks in a single call, instead of a single block per call, avoiding a lot of overhead. This improves the performance, and it's particularly useful with delayed allocation (in the next point) and extents.

Delayed allocation

Delayed allocation, or lazy allocation, is a performance feature in which the writing of blocks to disk is done as late as permitted by memory constraints—that is, it delays block allocations from the *write()* operation time to the page flush time. This method provides multiple benefits:

- It increases the opportunity to combine many block allocation requests into a single request, reducing fragmentation and saving CPU cycles.
- It avoids unnecessary block allocation for short-lived cycles.

Consider a hypothetical scenario in which you are creating a file whose size increases monotonically during your program execution. In case of an eager memory allocation system, every time you do a *write()*, it will allocate blocks on disk and it is very possible that

even for the same file, the blocks are getting allocated in a fragmented manner. Under a similar scenario, since delayed memory allocation will wait till a page flush, chances are that large chunks of the file will be allocated in contiguous locations on disk, thereby decreasing fragmentation.

Extents, the multiple block allocator and delayed allocation together result in better performance and reduced fragmentation when ext4 is used.

Fast fsck

Ext4 adds data structures that permit *fsck* to skip unused parts of the disk in its checks, thus speeding up file system checks.

Persistent preallocation

Often, applications might need 'guaranteed' disk space before they start their execution. Thus, they would need to reserve or pre-allocate some space so that they don't run out of it during execution. ext4 makes this possible via the *posix_fallocate()* libc interface (www.opengroup.org/onlinepubs/.../posix_fallocate.html). This feature was available in later versions of ext3 as well.

Nanosecond timestamps

Nanosecond timestamps mean that inode fields like 'modified time' will be able to use the nanosecond resolution instead of the second resolution of ext3.

ext4 also brings some more miscellaneous features, such as 'No Journaling' mode and Journal Checksumming. Online defragmentation and 'undeletion' of files are future features, which are not yet production ready.

Using ext4

There is more than one way in which you can start using ext4 on your Linux distribution, if you are not using a recent release of Ubuntu or Fedora.

New ext4 FS from scratch

This is the easiest way to start using ext4 file systems. Create an ext4 filesystem using *mkfs.ext4* as you would for any of the other filesystems. If you get a 'command not found' or a similar error, you need an update of your *e2fsprogs* (<http://e2fsprogs.sourceforge.net/>). If you have some disk space lying free, create an ext4 partition and start using it without disturbing your current operating system set-up.

Migrate existing ext3 FS

The *tune2fs* utility can be used to 'turn on' the ext4 features on an ext3 filesystem. For example, the following command:

```
# tune2fs -O extents,uninit_bg,dir_index,inode100
```

...turns on support for extents, uninitialised block